L44 — File Organization: Sequential, Relative, Index Sequential, Inverted

Unit: 3 | **Chapter:** File Organization | **BT Level:** BT6 | **CO:** CO2, CO4

Learning Objectives

- Define file organization and explain its impact on access efficiency
- Compare sequential, relative (direct), index sequential, and inverted file organizations
- Identify the appropriate organization for a given use case
- Implement basic access patterns for each organization type

---

1. What is File Organization?

File organization is the method by which records are arranged within a file for storage and retrieval. The choice affects:

- **Access time** — how quickly records are found
- **Storage efficiency** — space used for data vs overhead
- **Update efficiency** — cost of insert, delete, modify

---

2. Sequential File Organization

Records are stored in a physical sequence — one after another, typically sorted by a key field.

```
Record 1 | Record 2 | Record 3 | Record 4 | Record 5
  id=101    id=105    id=110    id=115    id=120
```

Access Pattern

```
def sequential_search(file_records, target_id):
    """Linear scan through sorted records."""
    for record in file_records:
        if record['id'] == target_id:
            return record
        if record['id'] > target_id:   # Sorted file — can stop early
            break
    return None

def sequential_range_query(file_records, low_id, high_id):
    """Find all records with id in [low_id, high_id]."""
    result = []
    for record in file_records:
        if low_id <= record['id'] <= high_id:
```

```
            result.append(record)
        elif record['id'] > high_id:
            break
    return result
```

Characteristics

Feature	Sequential File
Search	O(n) linear scan
Sorted search	O(log n) binary search
Insert	O(n) — must shift records
Delete	O(n) — mark deleted, periodic compaction
Range query	O(k + log n) — k matching records
Best for	Batch processing, reports, backup

Use cases: Payroll processing, sorted reports, log files, magnetic tape systems.

---

3. Relative (Direct) File Organization

Records are stored and accessed by their **relative record number (RRN)** — a fixed position in the file. Record i is at offset `i × record_size` bytes.

```
Offset 0:  Record 0
Offset R:  Record 1
Offset 2R: Record 2
...
Offset iR: Record i
```

Access Pattern

```
import struct

RECORD_SIZE = 100  # bytes per record

def read_record(file_path, rrn):
    """Direct access by relative record number. O(1)"""
    with open(file_path, 'rb') as f:
        f.seek(rrn * RECORD_SIZE)
        raw = f.read(RECORD_SIZE)
    return raw

def write_record(file_path, rrn, data):
    """Write record at given RRN. O(1)"""
    assert len(data) == RECORD_SIZE
    with open(file_path, 'r+b') as f:
        f.seek(rrn * RECORD_SIZE)
        f.write(data)

# Hash-based direct access
```

```
def hash_record_rrn(key, table_size):
    """Map key to RRN using hash function."""
    return hash(key) % table_size
```

Characteristics

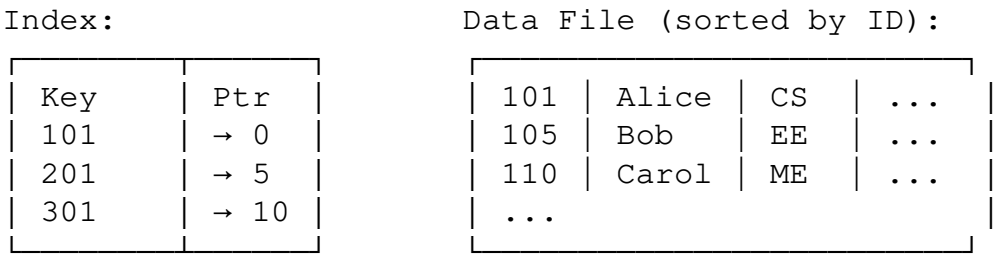
Feature	Relative File
Search by RRN	O(1) — direct seek
Search by key	O(1) with hash mapping
Insert	O(1) at known RRN
Delete	O(1) — mark as empty
Wasted space	Possible (empty slots)
Best for	Random access, real-time, phone directories

Use cases: Student records by roll number, employee records by ID, reservations by seat number.

---

4. Index Sequential File Organization (ISAM)

Two-level structure: sorted sequential data file + an **index** that maps keys to positions.



Access Pattern

```
import bisect

class IndexSequentialFile:
    def __init__(self):
        self.records = []   # Sorted list of records: [(key, data), ...
        self.index = []     # Index: [(key, record_position), ...]
        self.INDEX_INTERVAL = 5   # Create index every 5 records

    def build_index(self):
        """Build sparse index: one entry per INDEX_INTERVAL records."""
        self.index = []
        for i in range(0, len(self.records), self.INDEX_INTERVAL):
            key = self.records[i][0]
            self.index.append((key, i))

    def search(self, target_key):
        """Binary search in index → sequential scan in block. O(log n +
        # Find the index block containing target_key
```

```
        keys_only = [entry[0] for entry in self.index]
        block_start_pos = bisect.bisect_right(keys_only, target_key) -

        if block_start_pos < 0:
            return None

        _, block_start = self.index[block_start_pos]
        block_end = block_start + self.INDEX_INTERVAL

        # Sequential scan within block
        for i in range(block_start, min(block_end, len(self.records))):
            k, data = self.records[i]
            if k == target_key:
                return data
            if k > target_key:
                break
        return None
```

Characteristics

Feature	ISAM
Search	O(log n) index search + O(block_size)
Sequential scan	O(n)
Insert	Overflow area → can degrade over time
Delete	Mark deleted
Best for	Mixed random + sequential access

Use cases: Traditional banking systems, library catalog, airline reservations (legacy).

---

5. Inverted File Organization

Used for **full-text search** — the index maps **attribute values (words)** to the list of records containing that value.

```
Inverted Index (for document search):
"algorithm"  → [doc3, doc7, doc12]
"data"       → [doc1, doc3, doc5, doc9]
"structure"  → [doc1, doc5, doc11]
"tree"       → [doc3, doc5, doc7]
```

Build and Query Inverted Index

```
from collections import defaultdict

class InvertedIndex:
    def __init__(self):
        self.index = defaultdict(list)   # term → [doc_ids]

    def add_document(self, doc_id, text):
```

```
        """Index a document. O(|text|)"""
        words = text.lower().split()
        for word in set(words):   # set removes duplicates per doc
            self.index[word].append(doc_id)

    def search(self, term):
        """Find all documents containing term. O(1) lookup + result lis
        return self.index.get(term.lower(), [])

    def boolean_and(self, term1, term2):
        """Documents containing both terms."""
        list1 = self.search(term1)
        list2 = self.search(term2)
        set1, set2 = set(list1), set(list2)
        return list(set1 & set2)

    def boolean_or(self, term1, term2):
        """Documents containing either term."""
        list1 = self.search(term1)
        list2 = self.search(term2)
        return list(set(list1) | set(list2))


# Build and query
idx = InvertedIndex()
idx.add_document(1, "data structures and algorithms")
idx.add_document(2, "algorithms and complexity")
idx.add_document(3, "data mining and machine learning")

print(idx.search("algorithms"))         # [1, 2]
print(idx.boolean_and("data", "algorithms"))  # [1]
```

Characteristics

Feature	Inverted Index
Search by value	O(1) + result set
Boolean queries	O(
Build time	O(n × avg_fields)
Storage	O(n × fields) — larger than data
Best for	Full-text search, search engines, IR systems

Use cases: Google search index, database secondary indexes, library full-text catalogs.

---

6. Comparison Summary

Feature	Sequential	Relative (Direct)	ISAM	Inverted
Primary access key	Sorted key	RRN/hash	Key	Attribute value

Search	O(n) or O(log n)	O(1)	O(log n + block)	O(1)
Insert	O(n)	O(1)	Overflow handling	Rebuild/update
Space overhead	Minimal	Fixed slots	Index space	High
Best for	Batch, sequential	Random access	Mixed	Text/attribute search
Example use	Log files, payroll	Reservations, IDs	ISAM databases	Search engines

---

Summary

- **Sequential:** Simple, sorted — good for batch; slow for random access.
- **Relative/Direct:** O(1) access by position or hash — ideal for known-key access.
- **Index Sequential (ISAM):** Index accelerates random access while maintaining sequential structure.
- **Inverted:** Maps attribute values to record lists — ideal for full-text search and multi-attribute queries.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 11 (MIT Press, 2022)
- Ramakrishnan & Gehrke, *Database Management Systems*, Ch. 8–9 (McGraw-Hill, 2002)
- Manning, Raghavan & Schütze, *Introduction to Information Retrieval*, Ch. 1 (Cambridge, 2008)